

使用帮助手册 · USER GUIDE V1.0

VIT-Stego

轻量级 Vision Transformer 隐写分析工具箱：
从 BOSSbase 封面到可部署检测器的完整流程，
覆盖空间域与 JPEG 域（nsF5 / J-UNIWARD）、
弱负载与跨域迁移评测。本手册给出安装、数据
准备、训练、评估与命令行参考。

Python 3.10+ · PyTorch 2.x · CPU / GPU 单机 ·
2026 年 9 月

目录

1 项目简介	2
1.1 主要结果一览	2
1.2 两条关键结论	3
2 环境要求与安装	3
2.1 依赖清单	3
2.2 安装步骤	4
2.3 数据源与目录约定	4
3 空间域快速开始	4
3.1 第一步：生成数据集	5
3.2 第二步：训练检测器	5
3.3 第三步：评估与多 seed 基准	5
3.4 第四步：自检与汇总	5
4 JPEG 域 workflow	6
4.1 两种嵌入器	6
4.2 生成 JPEG 域数据集	6
4.3 在系数图上训练	6
4.4 结果解读	7
5 OOD / 弱负载 / 迁移评测	7
5.1 已验证的迁移规律	7
6 命令行参考	8
6.1 脚本总览	8
6.2 训练关键参数	8
7 代码结构	9
8 评测协议要点	10
9 常见问题	10

1 项目简介

VIT-Stego 是一个自包含、CPU/GPU 单机可跑的隐写分析研究工具箱：用紧凑的 Vision Transformer (LightViT, 9.6 万至 66 万参数) 区分封面图 (cover) 与含密图 (stego)。项目内置五种嵌入器——空间域的 HILL 自适应 ± 1 、HILL+STC、LSB 替换, 以及 JPEG 域的 nsF5 与 J-UNIWARD——并配套严格的多 seed 评测协议、SRNet 基线与 MAE 预训练。本手册面向使用者, 给出从环境安装到结果产出与迁移评测的完整操作说明。

1.1 主要结果一览

下表汇总空间域与 JPEG 域各工作点在 2000 张 BOSSbase 封面、held-out test 划分上的表现 (3 个随机种子均值 $\pm$ 标准差, 60 epochs)。全部数字可由仓库内命令复现, 记录存于 exp/ 目录。

输入	嵌入方式	参数量	test AUC	test acc
原始像素 (无 SRM)	HILL 自适应 ± 1 0.4bpp	478,657	0.5004 $\pm$ 0.0003	0.5004
SRM 残差	HILL 自适应 0.4bpp (默认 240K)	240,367	0.9036 $\pm$ 0.0188	0.8171
SRM 残差	HILL 自适应 0.4bpp (657K)	657,583	0.9094 $\pm$ 0.0086	0.8092
SRM 残差	LSB 替换 0.4bpp	657,583	0.7530 $\pm$ 0.0116	0.6687
SRM 残差	HILL + STC 0.4bpp	657,583	0.7507 $\pm$ 0.0158	0.6725
量化 DCT 系数图	nsF5 1.0bpnc Q75	120,833	0.9876 $\pm$ 0.0099	0.9408
量化 DCT 系数图	J-UNIWARD 0.4bpp Q75	120,833	1.0000 $\pm$ 0.0000	0.9962
JPEG 解压像素	J-UNIWARD 0.4bpp Q75	240,367	1.0000 $\pm$ 0.0000	1.0000

表 1 主要基准结果 (2000 封面, 3 seeds, 60 epochs)

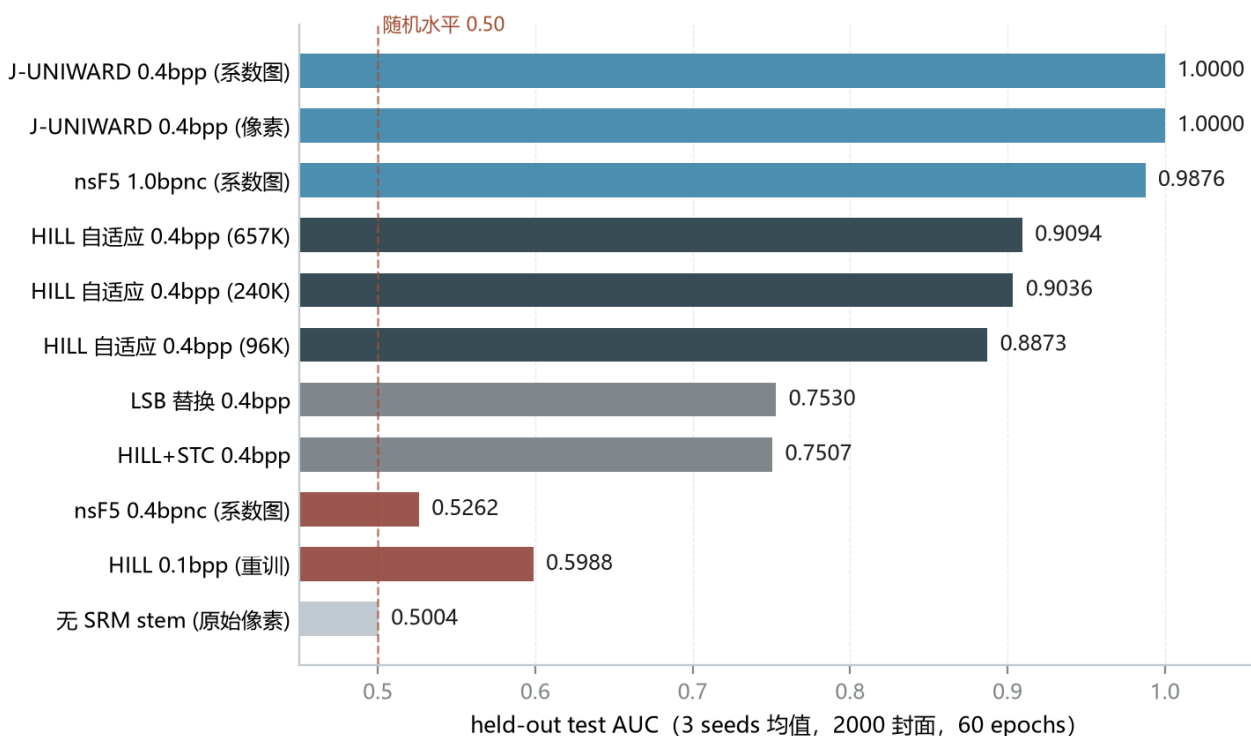


图 1 各工作点 held-out test AUC 总览 (3 seeds 均值)

1.2 两条关键结论

理解这两点是用好本工具箱的前提，也解释了默认参数为何如此设定。

- **残差 stem 是检测能否成立的关键。**纯 ViT 直接吃原始像素时恰好停在随机水平 (AUC 0.5004)；前置一个冻结的 SRM-30 高通残差 stem (JPEG 域则直接使用量化 DCT 系数图) 才能把 ± 1 量级的嵌入信号送到网络。
- **检测器要小不要大。**24 万参数的默认配置与 66 万参数版本在误差范围内不可区分，而参数量少 2.7 倍；9.6 万参数仍能保持接近 0.89 的 AUC。容量从约 19 万参数起进入平台期。

2 环境要求与安装

项目为纯 Python 实现，无网络依赖；所有产物（数据集、权重、实验记录）都写入项目目录，不污染系统盘。

2.1 依赖清单

核心依赖如下，建议 Python 3.10 及以上。

组件	版本要求	用途
Python	3.10+	运行环境

组件	版本要求	用途
numpy / scipy	任意近期版本	信号处理、嵌入器
PyTorch	2.0+ (CPU 或 CUDA)	训练与推理
scikit-learn	1.0+	AUC / accuracy 计算
Pillow / matplotlib	9.0+ / 3.5+	图像与图表
pytest (可选)	任意	仅在用 pytest 跑测试时需要

表 2 依赖组件

2.2 安装步骤

在项目根目录执行：

```
cd F:\VIT-stego
pip install -r requirements.txt
```

也可手动安装：pip install numpy scipy scikit-learn pillow matplotlib torch

2.3 数据源与目录约定

封面图取自姊妹项目的 BOSSbase HD 图像集，路径在 scripts/prepare_data.py 顶部的 BOSS_IMAGESET 常量中定义，可用 --imageset 参数覆盖。生成的一切数据落在项目内：

目录	内容	说明
data/	生成的 .npz 数据集	已 gitignore，可随时重建
models/	训练得到的权重与 sidecar JSON	已 gitignore，可重新生成
exp/	benchmark / OOD 实验记录	scripts/summarize.py 的输入
docs/	本手册与构建脚本	

表 3 目录约定

3 空间域快速开始

以默认的 240K 参数检测器为例，五个命令完成「数据 → 训练 → 评估 → 基准 → 自检」全流程。单张 GPU 上 60 epochs 单 seed 约 1–2 分钟，2000 张封面数据集生成约 3 分钟。

3.1 第一步：生成数据集

prepare_data.py 从图像集裁剪封面并按指定嵌入方式生成配套含密图，存为一个 .npz：

```
python scripts/prepare_data.py --n 2000 --size 256 --rate 0.4 \
  --method adaptive --out data/bossbase_hill_256.npz
```

--method 可选 adaptive (HILL 自适应 ± 1 , 默认) / stc (HILL+STC) / lsb (LSB 替换)

要点：--rate 是负载率 (bpp, 每像素嵌入比特数)；--size 是存储裁剪边长，训练时再从中随机取 128×128 视图，等效把每张封面扩成 64 个训练视图。

3.2 第二步：训练检测器

默认配置即 README 中的 240K 模型 (--srm --truncate 4 --pool mean 已是入口脚本默认)：

```
python scripts/train.py --data data/bossbase_hill_256.npz \
  --srm --truncate 4 --pool mean --epochs 60 --seed 0 \
  --out models/lightvit_lite_hill.pt
```

训练结束在权重旁写入同名词缀为 .json 的 sidecar 文件，记录全部超参、最佳 epoch 与 test AUC——后续 evaluate / eval_ood 会读取它，保证权重按训练时的配置被评估。

3.3 第三步：评估与多 seed 基准

评估已保存的权重，或跑真正应当引用的多 seed 数字：

```
python scripts/evaluate.py --data data/bossbase_hill_256.npz \
  --ckpt models/lightvit_lite_hill.pt

python scripts/benchmark.py --data data/bossbase_hill_256.npz \
  --srm --truncate 4 --pool mean --epochs 60 --seeds 0,1,2 --tag 2000_hill
```

上：评估 checkpoint (自动恢复 sidecar 配置)；下：多 seed 基准 (写入 exp/.json)

3.4 第四步：自检与汇总

仓库自带无需 pytest 的自检与记录汇总：

```
python tests/test_pipeline.py
python scripts/summarize.py
```

上：14 项不变量自检 (全部 PASS 才可信)；下：exp/*.json 渲染成 markdown 表 (含 OOD 表)

4 JPEG 域 workflow

真实部署中检测 JPEG 隐写时，检测器手里是 JPEG 文件本身。prepare_data.py 为此提供两种 JPEG 域嵌入器，并把「量化 DCT 系数图」与解压像素一并存入数据集，训练时用 --input coeff 选择系数图输入——此时无需 SRM stem（JPEG 量化本身即高通），模型只有 120,833 参数。

AUC 0.988 / 1.0000

nsF5 1.0bpnc 与 J-UNIWARD 0.4bpp 的系数域检测结果（Q75，2000 封面）

4.1 两种嵌入器

nsF5（非收缩 F5）把负载嵌入非零 AC 系数，奇偶翻转永不产生新零，接收端仅凭含密图即可确定载体集合；负载单位 bpnc（每非零 AC 的比特数）是文献标准约定，因为固定 bpp 并非对每张图都装得下（Q75 下每图非零 AC 数在 5.5k–33k 间波动）。J-UNIWARD 用 UNIWARD 小波代价驱动 STC，摊开在每块全部 63 个 AC 上，选点代价显著低于平均。

4.2 生成 JPEG 域数据集

一条命令同时得到解压像素封面/含密图与系数图（covers_c / stegos_c，DC 置零、AC 裁剪到 ± 30 ）。注意：J-UNIWARD 逐图约 1.6 秒，2000 张约需 1 小时；nsF5 全程仅数分钟。

```
python scripts/prepare_data.py --n 2000 --size 256 --rate 1.0 \
  --rate-unit bpnc --method nsf5 --quality 75 \
  --out data/bossbase_nsf5_256_q75_bpnc10.npz

python scripts/prepare_data.py --n 2000 --size 256 --rate 0.4 \
  --method juniward --quality 75 --out data/bossbase_juniward_256_q75.npz
```

上：nsF5（--rate-unit bpnc 或 bpp，此例为文献常用的 1.0 bpnc）；下：J-UNIWARD（负载单位固定为 bpp）

4.3 在系数图上训练

JPEG 域训练把 --input coeff 与 --pool mean 搭配，不再加 --srm：

```
python scripts/benchmark.py --data data/bossbase_nsf5_256_q75_bpnc10.npz \
  --input coeff --pool mean --epochs 60 --seeds 0,1,2 \
  --tag 2000_nsf5_q75_bpnc10_coeff_lite
```

coeff 输入要求 img_size 为 8 的倍数；与 --srm 互斥（脚本会直接报错）

4.4 结果解读

nsF5 在 0.4 bpnc 下每图平均仅改动约 1,271 个系数 (2%)，在本项目数据预算下两种输入都检测不了 (AUC 0.52–0.54，负结果如实记录)；负载升到 1.0 bpnc (约 8.8% 系数被改) 后系数域检测器达到 0.988。J-UNIWARD 0.4bpp 改动约 9,900 系数，两域输入均饱和到 1.0。弱负载部署的正确姿势见第 5 章的跨负载迁移。

5 OOD / 弱负载 / 迁移评测

scripts/eval_ood.py 对训练好的 checkpoint 不做任何重训，直接在其他数据集的 held-out test 划分上评分。所有数据集按相同顺序从相同封面构建，同一 seed 选出同一批源图，因此行间差异只归因于嵌入方式的变化。

```
python scripts/eval_ood.py --ckpt models/lightvit_lite_hill.pt \
  --tag lite_hill_ood \
  --data data/bossbase_hill_256.npz data/bossbase_hill_256_r02.npz \
    data/bossbase_hill_256_r01.npz data/bossbase_hill_stc_256.npz \
    data/bossbase_lsb_256.npz
```

结果写入 exp/json，并随 summarize.py 一起渲染

评测类型	目标数据	AUC	解读
分布内	HILL 自适应 0.4bpp	0.8994	基准
弱负载	HILL 自适应 0.2bpp	0.7548	平缓退化
弱负载	HILL 自适应 0.1bpp	0.6330	接近极限
嵌入器不匹配	HILL+STC 0.4bpp	0.7474	损失约 0.15
嵌入器不匹配	LSB 替换 0.4bpp	0.7489	同上
跨域	nsF5 JPEG 0.4bpnc	0.5315	空间域不迁移到 JPEG 域

表 4 HILL-0.4 训练模型的迁移评测 (exp/lite_hill_ood.json)

5.1 已验证的迁移规律

三条经验可直接指导部署：其一，空间域检测器不能迁移到 JPEG 域，检测器必须按域训练；其二，JPEG 域内部跨嵌入器同样不迁移 (J-UNIWARD 模型在 nsF5 上为 0.50)，检测器是嵌入器特异的；其三，唯一有效的正向迁移是「强负载预训练 → 弱负载部署」——1.0bpnc 系数模型直接用于 0.4bpnc 数据可得 AUC 0.7345，远高于在 0.4bpnc 上直接训练的 0.5262。

6 命令行参考

七个入口脚本共享同一引擎（src/engine.py），保证一次实验的 AUC 是模型的属性而非脚本的属性。常用参数如下，未列出的可用 --help 查看。

6.1 脚本总览

每个脚本一件事，输出落点固定：

脚本	功能	输出
prepare_data.py	裁剪封面 + 五种嵌入	data/<名>.npz (含 provenance 元数据)
train.py	单次训练，输出 test AUC	models/<名>.pt + 同名 .json sidecar
evaluate.py	按 sidecar 配置评估 checkpoint	终端 AUC/acc, 可 --dump-probs
benchmark.py	多 seed 基准	exp/.json
eval_ood.py	checkpoint 的跨数据集迁移评测	exp/.json
summarize.py	渲染全部实验记录	终端 markdown 表
diagnose_split.py	复现旧划分泄漏对照实验	终端对比曲线

表 5 入口脚本总览

6.2 训练关键参数

train.py / benchmark.py 共用的模型与数据参数：

参数	默认	说明
--model	vit	vit (LightViT) 或 srnet (SRNet 基线)
--srm / --truncate	关 / 0	加冻结 SRM-30 stem; truncate=T 把残差裁剪到 $\pm T$ (推荐 4)
--input	pixels	pixels=解压像素; coeff=量化 DCT 系数图 (JPEG 域, 与 --srm 互斥)
--pool	cls	cls=CLS token; mean=patch token 均值池化 (推荐)
--dim/--depth/--heads	64/2/4	240K 默认配置; 96K 用 32/2/2, 657K 用 96/4/3
--img-size / --patch	128 / 8	训练视图尺寸与 patch 边长
--epochs / --lr / --batch	30 / 1e-3 / 32	2000 封面建议 60 epochs

参数	默认	说明
--no-augment	关	关闭随机裁剪与翻转增强
--tta	关	评测时对 4 个翻转变体取平均（免训练涨点）
--rate-unit	bpp	prepare_data 专用：bpp 或 bpnc（nsF5）
--quality	75	prepare_data 专用：JPEG 质量因子

表 6 常用参数速查

7 代码结构

按「引擎 — 模型 — 嵌入器 — 入口 — 测试」五层组织：

路径	职责
src/engine.py	三段划分、成对同侧、训练循环、AUC 评测——所有入口共用
src/dataset.py	CoverStegoDataset：随机裁剪/翻转增强、确定性验证裁剪、像素/系数双输入
src/vit_stego.py	LightViT + SRMStem（冻结 SRM-30 前端）+ SDPA 注意力
src/srnet.py	SRNet 基线（瓶颈 ResNet，约 1.35M 参数）
src/mae.py	MAE 自监督预训练与迁移（实测无增益，保留备用）
src/srm_filter.py	SRM 30 个高通核（Fridrich）
stego/jpeg.py	8×8 正交 DCT 编解码、Annex-K 量化表、zigzag
stego/nsf5.py	nsF5：非零 AC 载体、非收缩翻转、自适应 Hamming 矩阵编码
stego/juniward_jpeg.py	J-UNIWARD 代价（db8 谱分解）+ 跨距 STC
stego/stc.py	伴式网格码：stride-1 与跨距两种调度，含暴力最优性自检
stego/hill.py	HILL 失真函数
scripts/	七个入口脚本（见第 6 章）
tests/test_pipeline.py	14 项不变量自检

表 7 模块一览

8 评测协议要点

本仓库的每个数字都建立在以下四条纪律上。复现或对比结果前请先确认遵守：

- **三段划分。** val 只用来挑最佳 epoch，test 在最后只评分一次并被上报。取「验证集跨 epoch 最大」是乐观估计——几百个验证样本下仅选择噪声就值好几个 AUC 点。
- **封面与含密图成对同侧。** 同一源图的 cover 与 stego 必须落在划分的同侧（engine.make_splits 已强制），否则近似重复样本泄漏到边界两侧，AUC 被内容记忆污染。
- **至少 3 个随机种子。** 单一划分自带约 ± 0.05 的纯噪声，benchmark.py 输出均值 $\pm$ 标准差与标准误，两个配置的差异要和标准误比。
- **保持原始量纲。** 像素停留在 $[0, 255]$ 、系数停留在量化值，任何 $[0, 1]$ 归一化都会抹掉 ± 1 量级的嵌入信号。

0.217 AUC

同样的权重、同样的 epoch，泄漏划分与正确划分的验证 AUC 之差
——评测协议值得较真

9 常见问题

按出现频率排列；其他问题可先跑 tests/test_pipeline.py 排除环境因素。

Q1：训练一次要多久？

GPU (CUDA) 上 2000 封面 60 epochs 单 seed 约 1–2 分钟，3 seeds 约 5 分钟；纯 CPU 约 4–7 倍耗时。数据生成：空间域三种嵌入每千张约 1 分钟，nsF5 2000 张约 3 分钟，J-UNIWARD 2000 张约 1 小时（1.6 秒/图）。

Q2：显存/内存占用多大？

默认 batch 32、128×128 输入、240K 参数模型，单卡 2 GB 以内即可运行；J-UNIWARD 生成阶段的内存峰值约 2 GB（256×256 代价图的批处理窗口汇聚）。

Q3：为什么我的 nsF5 数据集检测不到？

先确认两件事：训练用的是 --input coeff（系数图）而非解压像素；负载是否足够——0.4 bpnc 在 Q75 下每图平均只改约 2% 的系数，属于本数据预算下不可检测的工作点，请用 1.0 bpnc 或更高负载先验证管道。

Q4：如何新增一种嵌入器？

在 stego/ 下实现嵌入函数（输入封面数组，输出同形状含密数组）并在 prepare_data.py 的 --method 选项中注册；随后为它补一条「负载可恢复 + 改动幅度 ≤ 1 」的测试用例。若新嵌入器定义在系数域，同步让 prepare_data 输出 covers_c/stegos_c。

Q5: checkpoint 能否跨数据集直接用？

evaluate.py / eval_ood.py 会读取权重旁的 sidecar JSON 恢复全部超参，因此权重与数据集只要空间一致（同为像素域或同为系数域、img_size 不超过存储尺寸）即可直接评估。跨域迁移本身是否成立见第 5 章。

Q6: exp/ 里的记录格式是什么？

benchmark 类记录含 test_auc_mean/std/se、逐 seed 明细与完整 config；eval_ood 记录含逐数据集 AUC/acc 与 provenance (method/rate/domain/quality) 。summarize.py --filter <子串> 可按 tag 过滤。

Q7: 如何引用仓库里的数字？

一律引用 benchmark.py 产出的 test AUC (3 seeds 均值 $\pm$ 标准差)，并在表格标注数据集、负载率与 epochs；验证集最大 AUC 不应出现在任何结论里。

本手册对应仓库 F:\VIT-stego (Apache-2.0 License)。手册中的全部数字可由第 3、4 章命令复现，原始记录见 exp/ 目录；仓库 README 提供英文版完整说明。